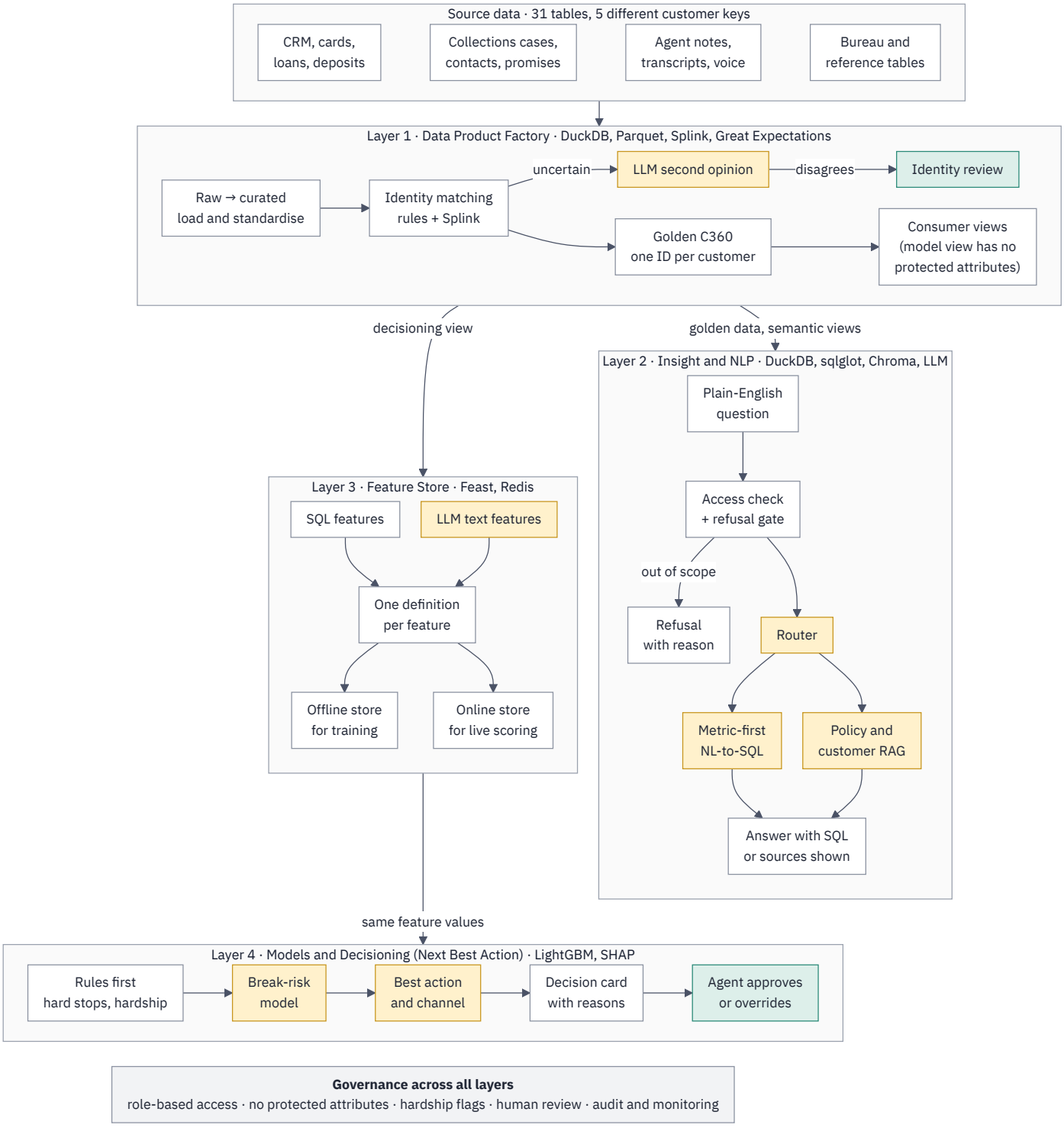
TrustIssues

Anushka Sinha, Aman Jayesh, Pranjal Garg, Shreyas Ramasubramanian



AI is used here A person reviews or approves

Design document

1. Summary

Problem. Collections teams make decisions on fragmented information, because each customer's accounts, payments, contacts and conversations sit in nine separate systems with five different customer keys, so treatment is one-size-fits-all, early signs of hardship are missed, and promises to pay are broken that could have been kept.

Solution. We unify this into one trusted Collections 360 record that teams can query in plain English and that powers a Next Best Action engine, which recommends the right treatment, channel and timing for every open case, explains why, and routes customers in hardship to specialist support.

Users. It serves collections agents who act on recommendations, team leads who monitor performance and approve actions, and the analytics team who own the metrics and models.

2. Business value

We improve the **promise-kept rate**: the share of promises to pay that are paid in full by the due date, as defined in the bank's certified metric definitions. We also track cure rate, roll rate from 1–30 to 31–60 days past due, and contacts per dollar recovered.

NBA moves the metric in three ways. It helps agents agree promises customers can actually keep, such as a date aligned to payday or a smaller instalment. It picks the best action before the due date for promises at risk. And it moves agent time away from customers who would pay anyway, toward those a call actually helps.

For the hackathon we compare our model with the bank's existing break-probability score on the same promises, using a time-based holdout: AUC, calibration, and the share of broken promises caught in the riskiest 20%. In production, NBA would run as a challenger strategy with a permanent 5% hold-out group, with complaints and opt-outs tracked as guardrails.

3. Layer 1: Data Product Factory

Data moves through three stages. **Raw** files are loaded exactly as received, with load time and source file recorded. The **curated** stage casts types and standardises the fields we match on: dates of birth, phone numbers, names, postal codes and employer names. The **golden** stage holds one record per customer, one per account, and a crosswalk from every source key to the golden ID. It all runs on DuckDB over Parquet on a single machine.

Matching customer IDs

Card, loan, deposit and bureau records each use their own customer key, none of them links to CRM, and CRM itself contains duplicates. Cases, contacts, promises and notes already point to CRM, so linking everything to a deduplicated CRM record connects the whole history. We match in passes, cheapest and most certain first:

Pass	What it does
Rules	Exact matches on strong combinations: national ID hash, name + date of birth + phone, or date of birth + name + postal code. A phone number alone is never enough.
Splink	Probabilistic scoring of the remaining pairs, compared only within blocks such as same date of birth and postal area. Above 0.95 links; below 0.80 stays separate.
LLM second opinion	Pairs scoring 0.80–0.95 are shown to an LLM as two tagged records. They link only if it agrees with Splink.
Human review	Any disagreement goes to a review queue, and the reviewer's decision is reused on every rerun.

When unsure, we keep records separate: a missed link can be fixed later, but a wrong merge shows a customer someone else's debt. Joint account holders are stored as relationships, never merged. Every golden field records

which source it came from, and safety flags such as cease contact, deceased, bankruptcy or open dispute are set if *any* source raises them.

From the golden record we publish one view per use: a customer timeline for question answering and the agent screen, semantic views for natural-language queries, and a decisioning view for the model that leaves out every protected attribute. AI helps by suggesting column mappings and drafting transformation SQL, which an engineer reviews, and by flagging unusual row counts or null rates on each load.

4. Layer 2: Insight and NLP

Every question passes a few checks before any AI runs. The user's role sets what they can see: agents only their assigned cases, team leads aggregates, analysts non-personal data. A small classifier screens for prompt-injection attempts, and a keyword gate refuses questions about protected attributes, bulk personal data or out-of-scope uses such as marketing. A refusal always explains why and suggests a question that can be answered. Relative dates like "last week" are turned into exact dates using the dataset's as-of date, not today's date.

A router then sends the question down one of three paths:

- **Data questions** are matched to a certified metric. The LLM only fills in filters and dates; the SQL comes from the certified definition. "What drove this?" questions split the metric by approved dimensions such as product, segment or stated reason for paying late, never by protected attributes. If no certified metric fits, the LLM writes SQL against about 15 relevant columns (out of 1,422), chosen by matching the question's words to the schema and finding the shortest safe join path. These answers are labelled "uncertified".
- **Customer questions** give the LLM that customer's full redacted history of notes and calls, in date order.
- **Policy questions** search the policy documents, filtered first to the version in force on the as-of date and to what the user's role may see, and answer with a quote and a citation.

Before any SQL runs, a validator checks that it only reads data, only uses allowed tables and columns, includes the metric's standard filters, and never selects personal or protected fields. Each answer shows what the system understood as editable filters, the SQL or sources it used, a short summary and suggested follow-up questions. If the sources don't contain the answer, it says so.

5. Layer 3: Feature Store

Feature	Source	Built with
Days past due, now and worst in the last 12 months	Account history	SQL
Promises broken in the last 90 days	Promises to pay	SQL
Gap between due date and payday; how late salary is	Salary credits	SQL
Card utilisation; deposit balance vs promise amount	Accounts	SQL
Change in bureau score over 90 days	Bureau	SQL
Response rate by channel, last 30 days	Contact history	SQL
Hardship signal; stated reason for paying late	Notes, call transcripts	LLM
Strength of the promise ("I'll try" vs "Friday, for sure")	Call transcripts	LLM
Dispute mentioned	Notes	LLM

Text and voice. Voice reaches us as call transcripts. The LLM reads only redacted notes and transcripts, and stores each signal with the sentence it came from, so agents can see the evidence. LLM features are checked against about 200 hand-labelled notes before they are used.

Same values for training and live scoring. Each feature is defined once in a Feast registry and computed by the same code in both places. For training, features are taken as they were at the moment each promise was made, so the model never sees the future. For live scoring, the same code writes the latest values to an online store after the nightly batch, and again when a payment or new promise arrives. Text features are extracted once per note and stored, so training and scoring read identical values.

6. Layer 4: Models and Decisioning

Rules first. Before any model runs, rules remove actions that aren't allowed: no contact for cease-contact, deceased, bankruptcy or open-dispute cases; only channels the customer has consented to; contact limits and permitted hours. Any sign of hardship or vulnerability sends the case to a specially trained agent, along with the hardship programmes the customer qualifies for.

Model. A LightGBM model predicts the probability that a promise will be broken. It is trained on past promises and tested on later months. Hardship signals can only make the recommended treatment gentler, never harsher, and automated tests check this.

Choosing the action. Each allowed action, including waiting, is scored by expected payment minus the cost of contact, using the bank's past test results. Actions are then fitted to the day's agent and channel capacity. A queue is ready each morning, and during calls a promise coach shows the agent the break risk of different dates or amounts.

Explanation. Each recommendation shows the top three reasons (from SHAP values), why it beat the next-best action, which rules ruled out others, and the note or call behind any text signal. For example: *"High risk of breaking (0.81): two promises broken in 90 days, salary 29 days late, note on 12 Sep mentions reduced shifts. Recommend a hardship call by a certified agent."* An LLM may reword this for readability, but a check rejects any wording that changes the facts.

Human review. Agents accept or override every call recommendation and give a reason. Team leads approve the daily batch of digital reminders. Hardship, vulnerability and high-balance cases always need approval. Overrides are reviewed weekly and feed back into the model.

7. AI usage

We use AI only where rules can't do the job, and every AI output is checked by a rule or a person.

Where	Why AI	How it's checked
Column mapping and SQL drafts	Speeds up mapping 1,422 columns	Engineer approves
Uncertain identity matches	Handles initials, nicknames, typos	Must agree with Splink, else a person decides
Understanding questions	Turns plain English into filters and dates	Certified SQL; validator before running
Policy and customer answers	Reads documents and notes	Must quote and cite, or say "not found"
Text features	Hardship and reasons live in free text	Tested against hand labels
Break-risk model	Reliable, explainable risk scores	Compared with the current model; fairness tests
Wording of recommendations	Easier for agents to read	Fact check; falls back to a template

No AI is used for refusals, access checks, SQL validation, hard stops or hardship eligibility.

8. Scalability

What	Mode	Target
Build the golden C360	Nightly batch	About 45 minutes
Features, text extraction, morning queue	Nightly batch	Ready before agents start
Re-score after a payment or broken promise	Near real time	Seconds to minutes
Promise coach during a call	Real time	Under 100 ms, no LLM
Data questions	Real time	1–2 seconds
Policy and customer questions	Real time	2–5 seconds

A single machine running DuckDB handles about a million customers. The LLM only sees small slices of the data (uncertain matches and new notes), which keeps cost and time down.

9. Governance

Access control. Agents see only their assigned cases, team leads see aggregates, and analysts see non-personal data. Personal details are masked by default, and policy documents are filtered by role before they are searched.

No protected attributes. The model's input view lists only the columns it allows, so new columns can't slip in. The 12 protected attributes in the data dictionary are left out, along with age and known proxies such as accent and interpreter use. They are used only to test whether outcomes are fair across groups.

Hardship flags. Hardship is detected from account flags and from notes and calls. A flagged customer always goes to a trained agent, and the flag can only lead to gentler treatment.

Human review and audit. Nothing that affects a customer happens without an agent seeing it first. Every answer and recommendation is logged with its sources and model version, and accuracy and overrides are monitored.

10. Build plan

In 36 hours we will build a working end-to-end demo: one golden record per customer across all sources, Ask answering the benchmark questions with the SQL or sources shown and out-of-scope questions refused, a trained promise-break model with explanations, and an agent screen to accept or override each recommendation.

Hours	What we build
0–4	Load all sources; set up certified metrics and the refusal gate; write the point-in-time training set; draft the text-feature prompt and hand-label about 200 notes.
4–12	Standardise and match customer IDs with rules and Splink; build the certified SQL path; compute the SQL features; set up policy search.
12–20	Add the LLM second opinion and publish the golden tables; add SQL validation and "what drove this" questions; train the model and compare it with the current score; build customer questions and the agent screen.
20–28	Run quality checks and the data quality report; test Ask on the benchmark questions; add explanations and decision rules; connect recommendation cards and the override log.
28–36	Run fairness tests, rehearse the end-to-end demo, record the video, finish the deck and submit by 9:00 PM on 4 Oct.

If time allows, we will add the in-call promise coach, near-real-time re-scoring after payments, and French-language policy answers.

Data contract: gold.c360_customer

The Collections 360 customer record: one row per customer, built from all source systems.

Ownership

Business owner	Head of Collections Data
Technical owner	Collections Data Engineering lead
Grain	One row per customer (golden ID)
Classification	Restricted: contains personal data

Refresh

Frequency	Daily, after source extracts land
Freshness	No more than 24 hours behind the sources
Publishing	Only if all blocking quality rules pass
Changes	Breaking changes agreed with consumers first

Schema

Field	Type	Description
golden_customer_id	string (key)	Format G-000000, stable across runs
source_keys	list	Every linked CRM, card, loan, deposit and bureau key
match_method, match_confidence	string, decimal	How the record was matched (rules, Splink, LLM, human) and its lowest link score
full_name, date_of_birth	string, date	Best available value, with the source it came from
segment, province	string	As held in CRM
consent by channel	boolean	Call, SMS, email, push, autodialer; most restrictive value wins
cease_contact, deceased, bankruptcy, open_dispute	boolean	True if any source says so
hardship_flag	boolean	Hardship recorded on any case or account
total_balance, total_overdue, max_dpd	decimal, int	Rolled up from all of the customer's accounts
worst_bucket, active_promise_status	string	e.g. 31–60; open, kept, broken
source_systems, refresh_ts	list, timestamp	Lineage and time of last refresh

Quality rules

#	Rule	If it fails
1	golden_customer_id is unique and never empty	Block publishing
2	Every source record maps to exactly one golden ID, or is flagged as unmatched	Block publishing
3	No automatic link below 0.95 match confidence unless confirmed by the LLM check or a person	Block publishing
4	No protected attributes or personal details in the model's decisioning view	Block publishing
5	A safety flag raised in any source is also raised here	Block publishing
6	Date of birth is valid and age is between 18 and 110	Warn and report
7	Customer totals equal the sum of their accounts	Warn and report
8	Data is no more than 24 hours old	Warn and alert

Allowed uses

- Collections operations and the agent screen
- Next best action scoring and recommendations
- Spotting hardship and offering support
- Collections reporting and analytics

Not allowed

- Marketing or cross-selling
- Credit decisions outside collections
- Any decision using a protected attribute
- Any customer action without human review